

Objectville II

SE 116 Project, 2025-2026 Spring

Project Description

Team Name: Just Coders

20250602221	Section 3	Melih Berk	Sönmez	melih.berk@std.izmirekonomi.edu.tr
20250602265	Section 3	Umut Utkan	Göbelek	umut.gobelek@std.izmirekonomi.edu.tr
20250602239	Section 3	Gökçe	Ünsal	gokce.unsal@std.izmirekonomi.edu.tr
20240602002	Section 3	Mert	Akalın	mert.akalin@std.izmirekonomi.edu.tr

GitHub Repository:
<https://github.com/iyiinsan5555/objectville-ii-se116>

Question 1

Which parts in your design and implementation did you use inheritance, polymorphism, and abstract classes and interfaces? State the reasons for using them.

In our project, we used several object-oriented programming concepts such as inheritance, polymorphism, abstract classes, and interfaces. These concepts helped us organize the code better and avoid repeating similar code in different classes.

One of the main examples of inheritance in our project is the *Cell* hierarchy. Since every object on the map has a location, we created an abstract class called *Cell* and stored common properties there. Classes such as *Zone*, *Road*, *Empty*, *ServiceProvider*, and *UtilityProvider* inherit from this class.

We also used inheritance in the zone structure. The *Zone* class is an abstract class and serves as the parent class for *Housing*, *Commercial*, and *Industrial* zones.

All zones share some common features such as level information, utility requirements, and output production. However, each zone type has its own production logic. For this reason, methods such as *calculateOutput()*, *update()*, and *updateLevel()* were declared as abstract and implemented differently by each subclass.

For example:

- Housing zones produce Population.
- Industrial zones produce Goods.
- Commercial zones produce Lifestyle.

We followed a similar approach for service providers and utility providers. The classes *Hospital*, *School*, and *PoliceStation* inherit from the abstract *ServiceProvider* class, while *PowerPlant*, *WaterPumpingStation*, and *InternetHub* inherit from the abstract *UtilityProvider* class.

Another concept we used was polymorphism. In several parts of the project, we stored different objects in collections using their parent type. Although these collections contain different zone types, we can process all of them through the same structure. Java automatically calls the correct version of a method depending on whether the “object” is a *Housing*, *Commercial*, or *Industrial* zone. This made our code cleaner and easier to maintain. The code example below (Figure 1.1) demonstrates the usage of polymorphism in our code. Where a zone object can be an *Industrial*, *Commercial*, or *Housing* instance. Whenever we call the *update()* method on that object, run-time polymorphism calls the correct implementation based on the object type.

```
//called in SimulationRunner
for (Zone zone : gridManager.getZones()) {
    zone.update(); //updating process depends on the object type
}
```

Figure 1.1 An example of the polymorphism usage

We also used the *Transferable* interface in our design. The *Zone* and *Road* classes implement this interface. Even though it is currently a marker interface (like *Serializable*), it helps identify classes that can participate in transfer-related operations (used in the BFS logic with the help of the *instanceof* operator). Using an interface gives more flexibility because other classes can implement it in the future without changing the existing class hierarchy.

Why These Design Choices Were Used

We used inheritance, polymorphism, abstract classes, and interfaces for several reasons:

- Code Reusability
- Reduced Code Duplication.
- Maintainability
- Extensibility
- High Cohesion and Low Coupling

Overall, these object-oriented concepts helped us create a more organized and reusable design. They also make the project easier to extend because new zone types, service providers, or utility providers can be added without major changes to the existing code. (Please check the Appendix A to see an example class diagram that we used in the development of the project.)

Question 2

Is your design open to extension for new types? Try to answer the following questions with respect to the submitted version of your code.

- If a new type of zone is introduced, say, “governmental” which requires population and produces “law” which will be used to create level 4 for zones, which parts of your code must change?*
- How about a new service cell type, such as “parks” which serves as “recreational” which parts of your code must change?*

Our design is fairly open to extension in the case of a new zone type because new zone types can be introduced by creating a subclass of *Zone* and implementing its own production and level-up behavior. For a new Governmental zone, we would create a new subclass and override the inherited methods related to production and zone updates.

However, because law would be a completely new resource type, some existing parts of the code would also need modification. The *ResourceDistributor* would need to be extended to track, accumulate, and distribute the new resource. Additionally, any zones that require law for advancement would need updates to their level-up conditions.

Therefore, adding a new zone type is straightforward due to inheritance and polymorphism, but introducing an entirely new resource requires modifications to the resource distribution system. This indicates that the design is open to extension for new zone types, although resource handling is not fully generic.

As for the new service cell type “Park”, which provides the recreational service, we would first create a new class as a subclass of *ServiceProvider*. This class would define its own service radius and constructor while reusing the inherited functionality provided by the superclass.

To integrate the new recreational service into the simulation, the existing distribution system can be extended with a small additional case in the service distribution switch statement. Finally, the *updateLevel()* methods of the relevant zone classes would need to be modified wherever recreational service is required for a level up.

Overall, adding a new service provider requires only minor changes to the existing codebase. The use of inheritance allows most functionality to be reused, making the design reasonably open to extension.

In conclusion, our design is reasonably open to extension. The use of inheritance and polymorphism allows new zones and service types to be added primarily through the creation of new subclasses, minimizing the need for major architectural changes. While introducing entirely new resource or service categories may require small modifications to distribution and level-up logic, the core structure of the program remains unchanged. This makes the system flexible enough to accommodate future additions while maintaining the existing design. (Please check the Appendix A to see an example class diagram that we used in the development of the project.)

Question 3

State where in your code you have used memory-resident structures or generic types.

In our project, as data structures we utilised ArrayLists, Queues, HashSets and as their respective generic type parameters used:

<Cell>, <Zone>, <UtilityProvider>, <ServiceProvider>, <Point>, <PowerPlant>, <WaterPumpingStation> and <InternetHub>

MapReader Class

`ArrayList<Cell> cellArrayList` — By using the polymorphic base class Cell, this list can store any grid element type (Zones, Service Providers, Utility Providers, Roads, and Empty cells) within a single contiguous block of memory. Downstream classes then filter this root array into type-specific collections based on their requirements.

GridManager Class

To avoid iterating over the entire grid during type-specific simulation steps, GridManager downcasts and filters the master `cellArrayList` into type-specific generic lists. The `cellArrayList` elements are separated into designated arrays based on their runtime type:

- `ArrayList<Zone> zones` — stores Housing, Commercial, and Industrial objects
- `ArrayList<UtilityProvider> utilityProviders` — stores InternetHub, PowerPlant, WaterPumpingStation
- `ArrayList<ServiceProvider> serviceProviders` — Hospital, School, PoliceStation

These collections are shared directly with downstream subsystems via their constructors, so each subsystem can perform their specialized calculations directly

Within the same loop using `cellArrayList` a hashmap is constructed `HashMap<Point, Cell> cellHashMap` — This map directly links a coordinate object (Point) to its corresponding Cell and helps simplify the `getNeighbors()` method.

We chose a HashMap over a 2D array or separate child arrays for the following reasons. While a 2D array requires fixed indexing and nested loops to traverse, a hashmap provides $O(1)$ constant-time lookups by coordinate. It eliminates the need for chained method calls to retrieve coordinates during tick operations. This implementation significantly simplifies neighbor checking and type sorting. (Please check references [1], [2], [3], [4] to fully understand the HashMap and HashSet implementations.)

`ArrayList<Cell> neighbors` — Used inside the `getNeighbors()`

This list contains adjacent cells for a given coordinate. It utilises the convenience of HashMap structure, since each neighbor is retrieved in $O(1)$ time by looking up at the Point key directly.

Note that we used “Euclidean Distance” in the GridManager class.

UtilityDistributor Class

Receives the following from its constructor:

- `HashMap<Point, Cell>` — the shared hashmap for BFS traversal
- `ArrayList<UtilityProvider>`

It further distills the `ArrayList<UtilityProvider>` into type-specific lists:

- `ArrayList<PowerPlant>`
- `ArrayList<WaterPumpingStation>`
- `ArrayList<InternetHub>`

To execute Breadth-First Search (BFS) during utility propagation, it also initializes:

- `Queue<Cell>` — holds cells queued for exploration in BFS order
- `HashSet<Cell>` — tracks already-visited cells to prevent reprocessing
- `ArrayList<Cell> neighbors` — collects adjacent connectable cells at each BFS step

ServiceDistributor Class

Receives the following from its constructor and uses them for radius-based checks:

- `ArrayList<Zone> zones` — iterated over to retrieve the location of each zone
- `ArrayList<ServiceProvider> serviceProviders` — iterated over to retrieve each provider's location and radius

ResourceDistributor Class

Receives `ArrayList<Zone> zones` from its constructor

Calculates total global sums, divides them to zones, and modifies the internal fields of each zone (e.g., population and lifestyle levels).

Question 4

State where in your code you have used used lambdas or packaged units.

In our project implementation we did not utilise lambda expressions or functional interfaces, prioritising readability and ease of debugging among our group members. Our simulation logic relies explicitly on loops and method calls, which we found clearer to manage and navigate through for the scale of this project.

For packaged units, the project is organised under the root package `com.objectville`, divided into the following subpackages:

- `core`,
- `entity.cells`,
- `entity.cells.zones`,
- `entity.cells.serviceProviders`,
- `entity.cells.utilityProviders`,
- `fileIO`,
- `interfaces`,
- `util`.

Each subpackage groups classes by their role in the simulation, keeping entity definitions, utility logic, and core simulation classes separated from one another. (To see a representation of the package system, please check Appendix B.)

Question 5

Describe how your code handles the map files to create the map in your program.

For the sake of high cohesion and low coupling, we decided to create a class called "MapReader" to handle reading operations. The MapReader class has 2 methods, which are:

- 1.) `public ArrayList<Cell> readMap(Path mapPath)`
- 2.) `private Cell createCellFromSymbol(char symbol, int x, int y) throws IllegalArgumentException`

In the *readMap* method,

We used `BufferedReader` to read the given map file line by line.

As each line is read, we get every symbol (character) in that line and pass the symbol along with coordinates (x, y) to the *createCellFromSymbol* method to create the corresponding cell iteratively.

In the creation of the MapReader class,

We tried to apply the pragmatic programming principles that are taught in week 9. We used constants (static & final) instead of literals, used consistent naming, and good exception handling.

In the exception handling,

We decided that it would be a good idea not to crash the system unless there is a huge problem, such as a file that cannot be found. So by default, if there is an unknown symbol such as 'Ü', we created an Empty type cell.

After all the cells are read and added to the `ArrayList`, the *readMap* method returns the `ArrayList` and hands control to the others.

Question 6

Write a postmortem section. A postmortem discusses what went right and what went wrong in your project. Did you come across any common pitfalls? If so, how did you recover? If not, what did you do to not fall into these pitfalls?

In the first phases of the development, we thought it would be a good idea to create classes for services, resources, and utilities. Check the Figure 1.2 to see the design. (In fact, we created some of them: resources, services)

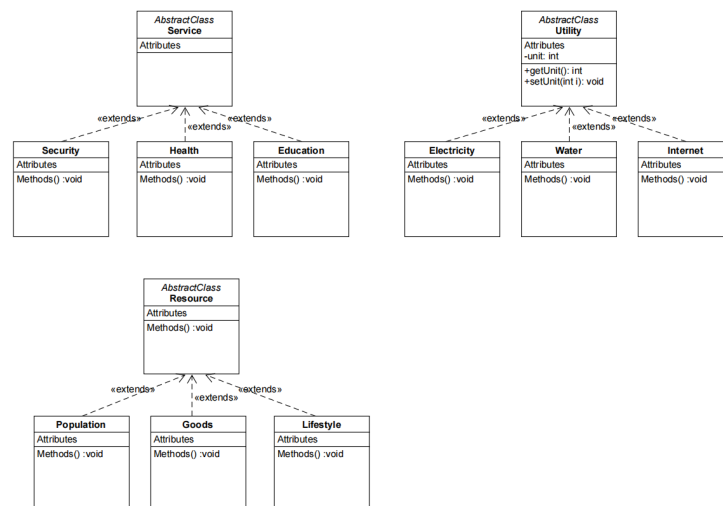


Figure 1.2 Our wrong class diagram

Unfortunately,

Instead of making our job easier, this class design made our job way harder. This is the time we learned that “a good design must be balanced, avoiding excessive modularity”.

We also had difficulty using Git and with communication among group members. Even though we gave ourselves time to learn Git, some of the team members were struggling to use Git. With the help of further research and face-to-face conversations with group members, we overcame this challenge too.

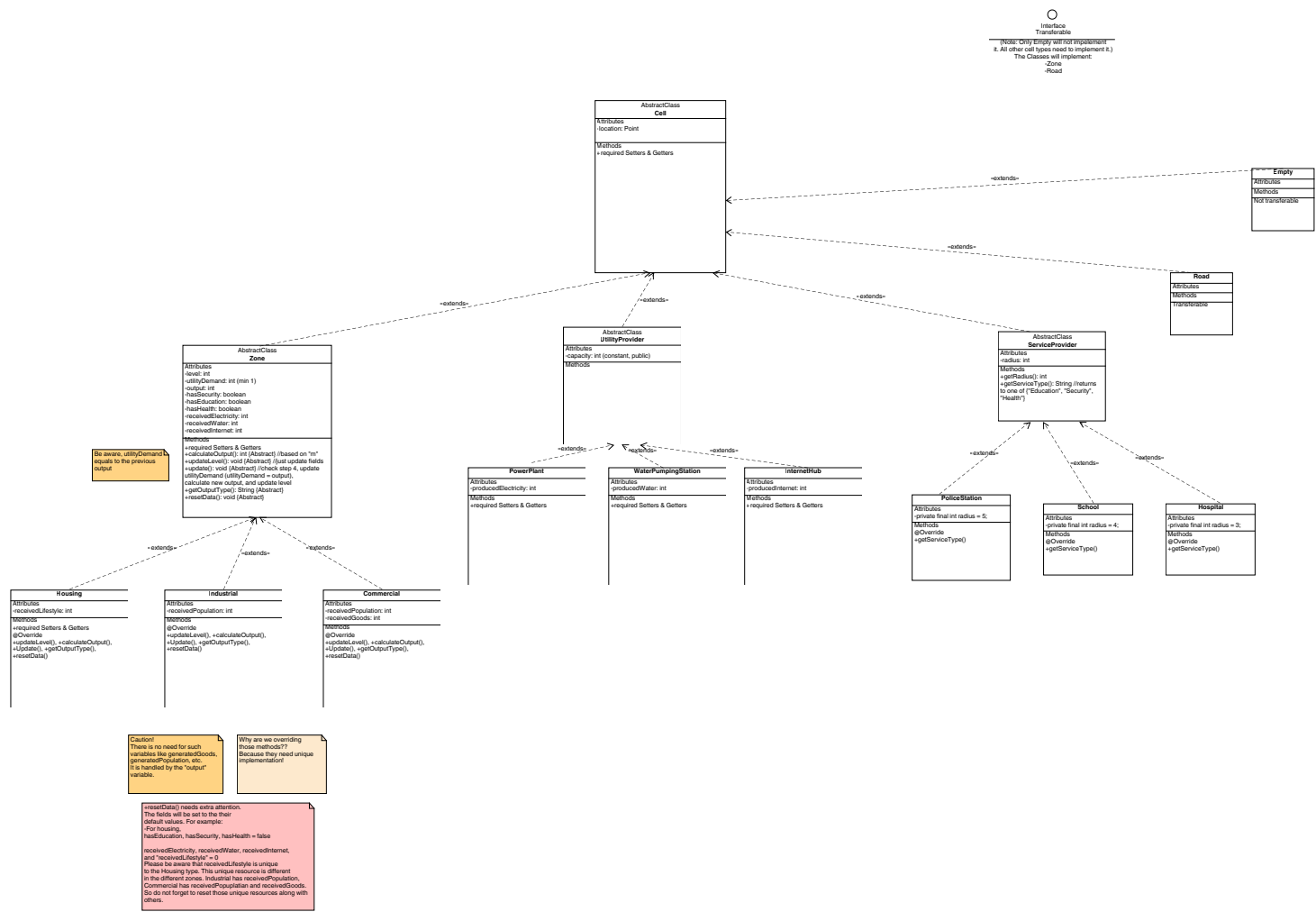
Despite some design issues, we think we did a good job on the project because we tried to follow the software development principles that we learned in CE112, as indicated below:

Requirement Analysis ↔ Design ↔ Implementation ↔ Testing

Agile Methodology

Additionally, we think that we followed object-oriented software architecture principles that are taught in week 8, such as modularity, low coupling, high cohesion, and packaging. Thus, we did not have much trouble.

Appendix A: System Architecture Diagram (Last Version)



SimulationRunner

It will iterate through a for loop (tick amount) the for loops consists of steps:

Step 1: ServiceDistributor distribute()

responsibilities: reset every provided service, distribute the services (if they are in range of the provider (security, health, education), each cell will have: hasSecurity, hasEducation, hasHealth: boolean and the distributor will change those variables.

Step 2: UtilityDistributor distribute()

Responsibilities: distribute utility via BFS. The cells will have: receivedElectricity, receivedWater, receivedInternet: int. Also utilityProviders will have (constant) capacity and produce(Elec., Water, ...) In BFS you will decrease this produced units and distribute until the utilities are fully consumed or cells (that consumes) are finished.

Step 3: ResourceDistributor distribute()

Responsibilities: Will pool (int division) among the relevant zone. The relevant zones will have receivedGoods, receivedElectricity, receivedPopulation: int. But be aware commercial zones need receivedGoods attribute but do not need others. (Be aware of this while creating new fields)

Step 4: Update zones: Zone.update()

What will it do: Zones will produce output (create relevant fields) Levels will be updated (Do not forget to use methods inside the Zone) Lastly, clean up the data inside the zones. (Make sure to clean only designated variables, we need some of them like Output for the future work)

Step 5: ResourceDistributor accumulate()

Accumulates the new production. Please create the relevant fields inside the ResourceDistributor. And loops like that

Make sure to declare fields that will be reached by a subclass or a class that is in the same package "protected".

GridManager

Attributes: cells: HashMap<Point, Cell>, cellsArray: ArrayList<Cell>, zones: ArrayList<Zone>, utilityProviders: ArrayList<UtilityProvider>, serviceProviders: ArrayList<ServiceProvider>

Methods: +required: getters

Things to add: The GridManager object will be created inside the simulation runner.

ServiceDistributor

Attributes: zones: ArrayList<Zone>, serviceProviders: ArrayList<ServiceProvider>

Methods: +distribute(): void (distribution logic that is mentioned in the Step 1)

Constructor: ServiceDistributor(ArrayList<Zone> zones, ArrayList<ServiceProvider> serviceProviders)

The logic: For every service provider, iterate through the zones and if the zone is in the area of the service provider change its fields to provide the service. (Service type is determined by the ServiceProvider.getServiceType()) and the radius is determined by ServiceProvider.getRadius().)

UtilityDistributor

Attributes: cells: ArrayList<Cell>

Methods: +distribute(): void

Constructor: UtilityDistributor(ArrayList<Cell> cells, ArrayList<Cell> cellsArray)

It is hard to plan BFS so the person who makes it (UTRAN) will design this class. What we need? After UtilityDistributor.distribute() method all the distribution logic must be done. We do not care about the internal structure of this class. Also check the step 2.

ResourceDistributor

Attributes: totalPopulation: int, totalGoods: int, totalLifestyle: int, totalHousing: int, totalCommercial: int, totalIndustrial: int, cellsArray: ArrayList<Cell>, zones: ArrayList<Zone>

Methods: +distribute(): void (distribute based on the logic. Use setters to set the received resource for the zones. (Make sure to check step 3 & project description), +accumulate(): void (make sure to reset relevant attributes (totalPopulation etc.) and then iterate through all zones and sum their outputs (Calculated in the step 4))

Constructor: ResourceDistributor(ArrayList<Cell> cells, ArrayList<Zone> zones)

Also initialize the attributes to 0.

We will create some all classes like ServiceDistributor, ResourceDistributor, UtilityDistributor, Why? Because those classes will oversee all cells and it will be easier for us to manipulate data like that.

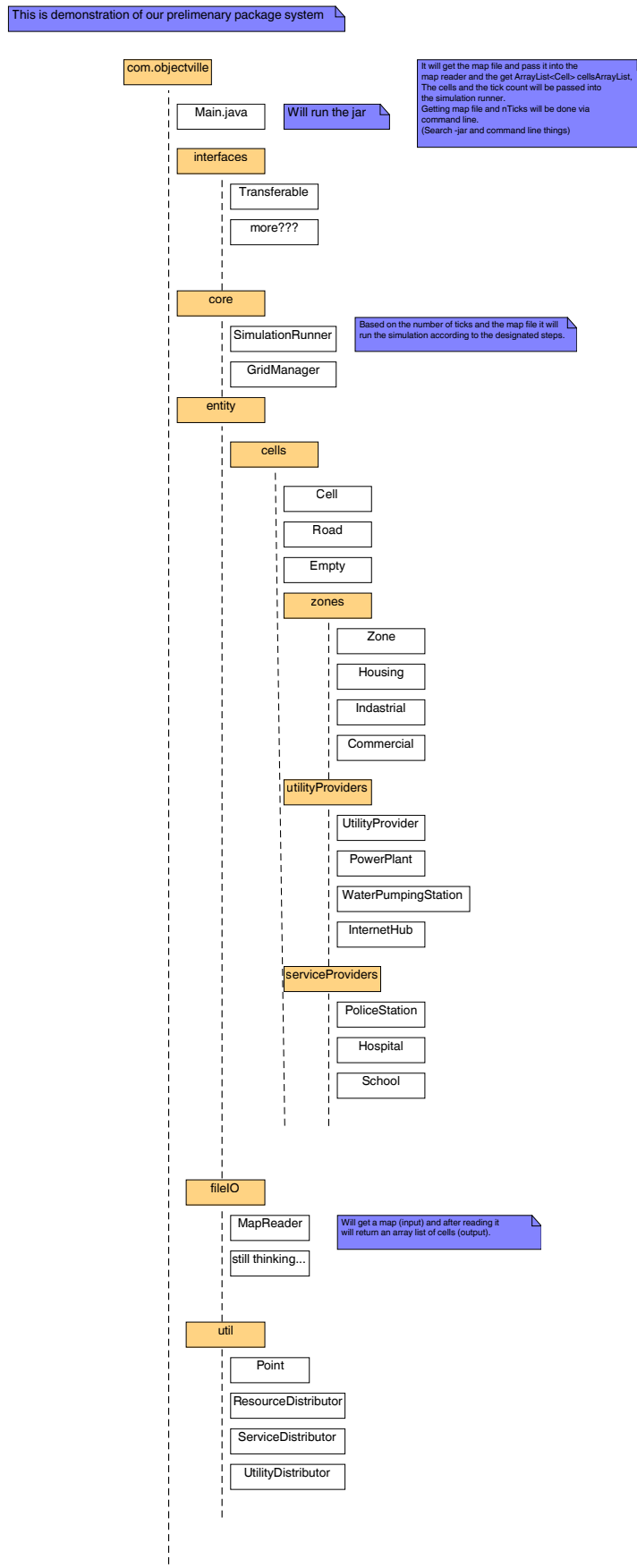
Note for Developers

You can create local variables to implement the desired logic. It is up to you. Others will not care how you implement the logic (abstraction!). But never forget to apply the rules that we learned in SE116 lecture 9 (Pragmatic Programming!)

For now.

We have deleted the levelable interface. We might add it later but at the moment, we do not think it is necessary. In addition(), The "Transferable" interface now will be like "Serializable" so it will not have any methods or fields. We are doing it for sake of simplification.

Appendix B: A Representation of The Package System



References

[1] Oracle. "Object (Java Platform SE 8)." docs.oracle.com, Accessed 17 May 2026.

<https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html#equals-java.lang.Object->

[2] Oracle. "Object.hashCode() (Java Platform SE 8)." docs.oracle.com. Accessed 17 May 2026.

<https://docs.oracle.com/javase/8/docs/api/java/lang/Object.html#hashCode-->

[3] Oracle. "Objects.hash() (Java Platform SE 8)." docs.oracle.com. Accessed 17 May 2026.

<https://docs.oracle.com/javase/8/docs/api/java/util/Objects.html#hash-java.lang.Object...->

[4] Bro Code. "Learn Hash Tables in 13 minutes #." YouTube video, Uploaded October 20, 2021. Accessed 17 May 2026.

<https://www.youtube.com/watch?v=FsfRsGFHuv4>

[5] [GeeksforGeeks](https://www.geeksforgeeks.org/). "Breadth First Search or BFS for a Graph."

GeeksforGeeks.org. Accessed 10 May 2026.

<https://www.geeksforgeeks.org/breadth-first-search-or-bfs-for-a-graph/>